

Project Understanding Guide — RF Modulation Classifier

What this document is for: so that when someone asks "why did you do it this way," you have a real answer rather than a memorized one.

What it is not: a replacement for the Tier 1 curriculum (3Blue1Brown, Loeber's PyTorch playlist, Velardo's FFT/STFT material). This explains how those concepts connect to *this specific project*. It doesn't teach them from scratch.

Part I is domain and technology background. Part II walks through each of the eight exercises you'll implement yourself — what it's testing, how to approach it, and what the interviewer behind it is really asking.

PART I — Background

1. What modulation classification actually is

Radio signals carry information by varying some property of a carrier wave — amplitude, phase, frequency, or some combination. Different modulation schemes (BPSK, QPSK, 16-QAM, FM...) encode bits differently. A receiver that doesn't already know which scheme is in use can't demodulate the signal.

Why this matters for SIGINT: an adversary's transmitter will not tell you what modulation it's using. Automatically classifying the modulation of an intercepted signal, with no prior coordination, is a standing problem in electronic warfare and signals intelligence. That's your answer to "why does this project matter" — and it's a real answer, not a stretch.

The classical approach uses hand-engineered features: cyclostationary feature detection, higher-order statistical moments of the signal, decision trees over those features. It works, and it's still deployed. It's also brittle — it needs an expert to design features per modulation type, and it degrades badly at low SNR.

The learned approach treats it as supervised classification: feed the signal to a neural network, let it learn discriminative features. This is the same transition image classification made from SIFT/HOG features to CNNs. Being able to articulate *both* approaches, and why the learned one is not automatically better, is a stronger position than treating the CNN as obviously correct.

2. IQ data — what you're actually feeding the network

A radio signal is physically a real-valued voltage varying over time. The receiver does not usually hand you the raw RF waveform as a single scalar sample. Instead, it downconverts the signal to baseband and represents each sample as a complex number:

$$s(t) = I(t) + jQ(t) = A(t)e^{j\phi(t)}$$

where:

- $I(t)$ is the in-phase component,

- $Q(t)$ is the quadrature component,
- $A(t)$ is the instantaneous amplitude,
- $\phi(t)$ is the instantaneous phase.

This is where Euler's identity comes in:

$$e^{j\phi} = \cos \phi + j \sin \phi$$

So the “imaginary” part is not a separate mystery quantity. It is a coordinate system that lets you represent both amplitude and phase in a single compact object. In practice, that means one complex sample carries two orthogonal real-valued pieces of information.

Why two numbers instead of one? Because a single real amplitude sample cannot distinguish phase. A phase shift is exactly what modulation schemes like BPSK, QPSK, and QAM use to carry information. Those phase changes show up as rotation in the I/Q plane.

A good mental model is this:

- the sample is a point in the plane,
- the point's x-coordinate is I ,
- the point's y-coordinate is Q .

If you collect many samples and plot them as Q versus I , you get a constellation plot. That plot is the geometric view of the modulation.

- QPSK uses four valid symbol locations, so a clean QPSK example should look like four clusters.
- 16-QAM uses sixteen valid symbol locations, so a clean 16-QAM example should look like sixteen clusters.
- Noise makes those clusters fuzzy, but the constellation geometry is still the same.

That is the core intuition behind the whole project: the network is not “seeing magic IQ.” It is seeing a time-ordered sequence of complex samples whose geometry encodes the modulation scheme.

RadioML 2018.01a gives you 1024 (I, Q) pairs per example, across 24 modulation classes, at SNRs spanning roughly -20 dB to +30 dB.

3. Input representation: raw IQ vs. spectrogram

Option A — raw IQ into a 1D CNN. Treat the sequence as 2 channels $\times$ 1024 timesteps. `nn.Conv1d` slides learned filters along the time axis.

Option B — spectrogram into a 2D CNN. Apply an STFT to get a time-frequency image, then treat it as image classification with `nn.Conv2d`.

The recommendation is **A**, for three reasons: it's simpler, it's what the original DeepSig/O'Shea work did, and it introduces one fewer transform you have to justify. Option B is a reasonable *comparison* to run later, not a default.

The tradeoff worth being able to state: the STFT throws away some information (specifically, the standard magnitude spectrogram discards phase) — and phase is exactly what several of these modulation schemes

encode information in. That's a real argument against Option B for this task, and it's a good thing to say out loud in an interview.

Where Velardo connects: if you go the spectrogram route, an STFT is a sequence of windowed FFTs. The window length controls a time/frequency resolution tradeoff — long windows give fine frequency resolution and coarse time resolution, short windows the reverse. This is the uncertainty principle showing up in signal processing. Velardo's series is framed around audio, but the math is identical. Skip his mel-spectrogram and MFCC sections — those encode assumptions about human hearing, which are irrelevant to RF.

4. Why a CNN works here

Modulation schemes have *local, translation-invariant* structure. A characteristic phase transition looks the same whether it occurs at sample 100 or sample 700. A convolutional filter learns to detect that pattern once and applies it everywhere — which is exactly the property that makes CNNs work on images (an edge detector doesn't care where the edge is).

Mechanically:

- **nn.Conv1d** slides a filter along the sample axis. Use **Conv2d** only on the spectrogram path.
- **Stacked conv layers with growing channel counts** build from low-level pattern detectors to higher-level combinations.
- **Pooling** shrinks the sequence while keeping strong activations, adding tolerance to small time shifts.
- **Fully connected head** maps learned features to 24 class logits.

3Blue1Brown builds this intuition for images. The transfer to 1D sequences is direct.

5. PyTorch syntax you'll actually use

- **nn.Module subclassing.** Layers in `__init__`, computation in `forward()`. Almost everything else builds on this.
- **Dataset / DataLoader split.** **Dataset** knows how to get *one* example (`__getitem__`, `__len__`). **DataLoader** handles batching, shuffling, and parallel loading. This split is why you can stream from a large HDF5 file without loading it into memory.
- **CrossEntropyLoss** applies softmax *internally*. Your model must output raw logits. Applying softmax in `forward()` and then using **CrossEntropyLoss** is a real, common, silent bug — it doesn't crash, it just trains badly.
- **loss.item()** detaches the scalar from the autograd graph. Accumulating **loss** instead of **loss.item()** keeps the entire graph alive across batches and leaks memory.

6. ONNX vs. TorchScript

A trained PyTorch model normally only runs inside a Python process with PyTorch loaded. To run it from C++, you need a portable artifact.

- **ONNX** is a framework-agnostic graph format. Run it with **ONNX Runtime**, which has a first-class C++ API and a much lighter dependency footprint than LibTorch.
- **TorchScript** is PyTorch's own serialization format, executed by **LibTorch** (PyTorch's C++ library). It handles PyTorch-specific ops that don't export cleanly to ONNX, but it's a heavy dependency to build against.

Lean: ONNX, primarily for the C++ dependency footprint. State the tradeoff explicitly rather than presenting it as arbitrary — that's the difference between "I picked ONNX" and "I picked ONNX because."

7. FFTW

FFTW ("Fastest Fourier Transform in the West") is a C library for FFTs. Its API separates **planning** (`fftw_plan_dft_1d` and friends — expensive, done once) from **execution** (`fftw_execute` — cheap, done many times). That split is a deliberate performance design: planning measures your hardware and picks an algorithm, so you amortize it.

Two things worth knowing that will come up: FFTW's plan creation is **not thread-safe** (execution of an already-created plan on distinct data is). And its plans are raw C resources requiring explicit destruction — which is precisely why Exercise 5 exists.

Check the current license terms yourself before use — FFTW has historically been GPL with a separate commercial license available, which matters if you ever intend to publish or ship the code. I'd verify this at <http://www.fftw.org/> rather than take my word for it.

8. ONNX Runtime's C++ API — the shape of it

The pattern: create an `Ort::Env` once, globally. Create an `Ort::Session` from your `.onnx` file plus `Ort::SessionOptions`. Per inference: wrap your input buffer in an `Ort::Value` tensor, call `session.Run(...)`, read the output tensor back.

Two gotchas that will bite you: `Ort::Value::CreateTensor` (the common overload) does **not** copy your data — it wraps a pointer you own, which must outlive the `Run` call. And the tensor's shape and memory layout must match what the model expects, in the order it expects.

Verify exact signatures against <https://onnxruntime.ai/docs/api/c/> at the time you write it. These shift between versions, and I'd rather you check than trust a signature I reproduced from memory.

9. Why p99 and not just mean latency

Mean latency hides tail behavior. A pipeline averaging 2 ms that occasionally takes 200 ms is a different system from one that always takes 2 ms, and in a real-time context the difference is the whole story. **p99 latency** — the value below which 99% of measurements fall — captures the tail the mean erases.

This is standard practice in latency-sensitive systems generally, and it's why reporting a bare mean in your write-up would be a visibly weaker choice. Report the distribution: mean, p50, p99, and ideally a histogram.

PART II — The Exercises

For each: what you're building, why *you* build it and not the tool, how to approach it, and the question behind the question.

Exercise 1 — `forward()`

What: the forward pass of your CNN. Input `(batch, 2, 1024)`, output `(batch, 24)` logits.

Why you write it: because the shape arithmetic through a conv stack is the architecture. If you can't compute what a `Conv1d(2, 64, kernel_size=7)` does to a tensor's shape, you don't understand the layer, you just know its name.

How to approach it:

1. Start with the dataset shape and make sure you know what each axis means:
 - axis 0 = batch,
 - axis 1 = channel (I and Q),
 - axis 2 = time/sample index. The raw input is therefore a sequence of 1024 complex samples, represented as 2 channels per time step.
2. Write the layers, but do not trust the model to be correct by inspection. Add `print(x.shape)` after each layer and run the code once. The goal is not to "make a pretty notebook" — it's to force the shape story into your head.
3. Work out the expected shape on paper first:
 - `Conv1d(2, 64, kernel_size=7)` keeps the batch dimension the same, changes the channel dimension from 2 to 64, and shortens the sequence length by padding/stride conventions.
 - A nonlinearity does not change shape.
 - `MaxPool1d` shrinks the sequence length.
 - The final `flatten()` must produce one vector per sample in the batch.
4. After the conv stack, check exactly where the dimension bug will appear. It is almost always the transition from the conv feature map to the classifier head.
5. Do **not** apply softmax. `CrossEntropyLoss` does it internally.

A concrete walk-through for this exercise:

- Build a toy `nn.Module` with one `Conv1d` layer, then one `ReLU`, then one `MaxPool1d`, then `flatten`, then `Linear`.
- Feed in a small batch and print the shape after every stage.
- Verify that the output of the last conv layer has the shape you expect before flattening.
- Confirm the flattened tensor is `(batch, features)` and not `(batch, sequence, features)`.
- Make the final linear layer output `(batch, 24)` logits.

The reason this exercise matters is that it turns the architecture from a list of names into a data-flow story. If you can explain what each layer does to the `[batch, 2, 1024]` tensor and why that transformation is appropriate for an IQ sequence, you are already answering the interview question in the right way.

The question behind it: "Walk me through your architecture." The bad answer names layers. The good answer says what each does to the data and why that's the right thing to do to an IQ sequence.

Exercise 2 — the training step

What: `zero_grad()` → forward → loss → `backward()` → `step()`.

Why you write it: this is the most-probed five lines of code in ML interviewing. It is also the place where the abstract idea of gradient descent becomes concrete.

How to approach it:

- Write it from memory first, then check. If you can't write it from memory yet, that's the signal to rewatch 3Blue1Brown's backprop episodes, not to look at the answer.
- Then, for each line, answer: *what breaks if I delete this line?* Delete it and find out. `zero_grad()` is the instructive one — without it, gradients accumulate across batches and your model trains on a nonsense signal, silently.
- **The overfit-one-batch test is the single most useful debugging tool in ML.** Take one batch. Train on it repeatedly. Loss must go to near zero. If it doesn't, your loop or your model is broken, and no amount of hyperparameter tuning on the full dataset will fix it. Run this before every full training run, forever.

The question behind it: "Your model isn't learning. What do you check?" The answer starts with "I'd overfit a single batch to isolate whether it's the loop or the data."

Exercise 3 — accuracy by SNR

What: group test predictions by SNR, compute accuracy per bucket, return sorted.

Why you write it: this plot is the most informative figure in your entire project. It's the one that shows you understand the difference between a number and a result.

How to approach it:

- You have `y_true`, `y_pred`, `snr` as parallel arrays. `np.unique(snr)` gives your buckets.
- Do not silently drop sparse buckets. Report their sample counts alongside their accuracy — an accuracy of 100% over 3 examples is not an accuracy of 100%.
- Then *look at the curve*. It should be near-chance ($1/24 \approx 4\%$) at -20 dB and rise steeply somewhere in the middle. If it's flat, something is wrong. If it's perfect at -20 dB, you have a leak.

The question behind it: "Your model gets 60% accuracy. Is that good?" The only honest answer is: it depends entirely on SNR, and here's the curve. Aggregate accuracy over a dataset with a wide SNR sweep is close to meaningless, and knowing that is the point of the exercise.

Also worth thinking about, and worth saying: some modulation pairs are *genuinely* indistinguishable at low SNR. There is an information-theoretic floor here. Your model failing at -20 dB may not be a model failure at all. Look at your confusion matrix at low SNR and see which classes collapse into each other — if they're the ones you'd predict on theoretical grounds, that's a much more interesting finding than a high accuracy number.

Exercise 4 — export verification

What: confirm the ONNX model's outputs match PyTorch's on real inputs.

Why you write it: because a silently-wrong export produces a C++ pipeline that runs beautifully and classifies garbage, and you will not notice for a week.

How to approach it:

- `np.testing.assert_allclose(torch_out, onnx_out, rtol=..., atol=...)`.
- **You must choose the tolerance and justify it.** Exact equality is the wrong assertion. Float32 arithmetic is not associative; two runtimes can reorder operations and get bit-different results that are both

correct. Something around `rtol=1e-3`, `atol=1e-5` is a reasonable starting point for float32 CNN logits, but derive your own and be able to say why.

- Better assertion than closeness of logits: do the two models produce the same `argmax`? That's what actually matters downstream.

The question behind it: "How do you know your deployed model is the model you trained?" Most candidates have never thought about this.

Exercise 5 — the FFTW RAI wrapper

(Only if you chose the spectrogram path.)

What: a class owning an `fftw_plan`. Constructor acquires, destructor releases, copy deleted, move implemented.

Why you write it: you're comfortable with modern C++, so this isn't a stretch — it's a *demonstration*. It's also the cleanest honest answer to "give me a concrete example of RAI preventing a bug."

How to approach it:

- Destructor calls `fftw_destroy_plan`. Straightforward.
- Copy constructor and copy assignment: `= delete`. Two objects must not own one plan.
- Move constructor: steal the pointer, **and null the source's pointer**. The moved-from object's destructor will still run. If it destroys the plan you just stole, you have a use-after-free that will pass every test until it doesn't.
- Move assignment: destroy your existing plan first, then steal. Handle self-assignment.
- The vector-reallocation test is not academic. `std::vector` moves its elements on growth. Get move wrong, and the crash surfaces there, far from the bug.

On thread safety: FFTW's *planning* is not thread-safe; executing an existing plan on separate data is. Decide whether your design cares, and be ready to say why. For a single-threaded benchmark harness the answer is "it doesn't, and here's why I know that" — which is a better answer than not having considered it.

The question behind it: "When have you used move semantics for something other than performance?" Here: for correctness of resource ownership.

Exercise 6 — the inference call

What: wrap a sample window in an `Ort::Value`, call `Run`, extract the predicted class.

Why you write it: the tensor layout and lifetime rules are where every real bug in this phase lives, and you only learn them by hitting them.

How to approach it:

- **Lifetime:** the common `CreateTensor` overload wraps a pointer you own without copying. The buffer must outlive `Run`. A tensor constructed from a temporary is a dangling pointer waiting for the right afternoon.
- **Layout:** row-major, in the order the model declares. If your predictions are wrong but *plausible-looking*, suspect layout before you suspect the model.

- **The test is the point.** Save the Python model's predictions for 100 test samples to disk. Assert the C++ pipeline reproduces them exactly. Not approximately — the argmax should match. If it doesn't, you have a plumbing bug, and the test tells you that immediately rather than three phases later.

The question behind it: "How did you validate the C++ path?" Having a real answer here is unusual and it will land.

Exercise 7 — latency instrumentation

What: time each inference, compute mean / p50 / p99.

How to approach it:

- **steady_clock, not system_clock.** `system_clock` can jump — NTP adjustments, DST — and a negative duration in your latency vector is a genuinely confusing afternoon.
- Record into a pre-`reserved` vector. Don't let the measurement apparatus allocate inside the thing you're measuring.
- **Decide what's inside the timer.** Tensor construction? Preprocessing? Just `Run`? All are defensible. Only one thing isn't: not knowing which you measured. Write it in the report.
- **p99 of 10 elements** is a real question with no canonical answer. Nearest-rank, linear interpolation, or refuse to report it below some N. Pick one, document it. An interviewer asking this is checking whether you noticed it was ambiguous.
- Warm up before measuring. First-call latency includes lazy initialization and cold caches; it's real, but it's a different number, and mixing it into your distribution corrupts both.

The question behind it: "What exactly did you measure?" Most benchmark numbers people quote don't survive this question.

Exercise 8 — one optimization, measured

What: hypothesize, implement one change, measure honestly.

How to approach it:

1. **Profile first.** Stage-level timings: I/O vs. preprocessing vs. inference. You cannot optimize what you haven't measured, and your intuition about where the time goes is probably wrong. Mine usually is.
2. **Predict before you measure.** Write down the number you expect. This is the single best calibration exercise in engineering, and almost nobody does it.
3. **Change one thing.** Two changes and a measurement is not an experiment.
4. Candidates worth considering: reusing the input buffer instead of allocating per sample; batching N windows per `Run()`; eliminating a copy between STFT output and inference input; `reserve()`ing the latency vector.

If it doesn't help: report that. "I hypothesized allocation overhead dominated, measured it, found inference was 94% of wall time, and the optimization moved p99 by 0.3%" is a *better* answer than a fabricated win. It demonstrates you measure rather than assume. And in an interview, the candidate who reports a negative result is the one who's actually run experiments.

Honest limitations — say these before you're asked

- This is a benchmark dataset, not a fielded system. The "streaming" in the C++ phase is a simulation over a static file. No live RF capture.
- Accuracy at low SNR will be poor, possibly near chance. That's expected and physically meaningful, not a bug — but you should present the SNR curve rather than an aggregate number.
- No FPGA. That's the separate Lariviere lab work. Keep the two stories distinct; conflating them weakens both.
- You did not beat state of the art, and shouldn't claim to. This is a well-studied task on a public dataset. The contribution is the end-to-end pipeline and the rigor, not the accuracy.

Leading with limitations is not modesty. It's the strongest available signal that you understand your own work.

Vocabulary

Term	Meaning here
IQ data	In-phase / quadrature components; complex baseband representation of the signal
SNR	Signal-to-noise ratio; varied per example in RadioML, and the axis your accuracy depends on most
STFT	Short-Time Fourier Transform; a sequence of windowed FFTs producing a time-frequency representation
Logits	Raw pre-softmax network outputs; what your model must return
ONNX	Portable, framework-agnostic neural network graph format
ONNX Runtime	The runtime executing ONNX models; has a C++ API
RAII	Resource lifetime bound to object lifetime; used here to own FFTW plans safely
p50 / p99	Median / 99th-percentile latency; the tail the mean hides
Overfit-one-batch	Debugging technique: if you can't drive loss to ~ 0 on a single batch, the bug is in the loop or model